Exploration Techniques and Intrinsic Motivation

```
When the unexpected knocks, curiosity opens the door.
```

Abstract

This topic delves into the exploration techniques and intrinsic motivation in reinforcement learning (RL), focusing on the critical role of exploration in discovering optimal policies in complex environments. It covers the distinction between epistemic and aleatoric uncertainty, the use of prediction error as an intrinsic reward, and the concepts of surprise and entropy. Advanced algorithms such as Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) are also explored. This topic equips students with the theoretical and practical tools to design RL agents capable of efficient exploration and learning in diverse settings.

Table of Contents

1. Curiosity and Exploration
    - Overview of curiosity in RL
    - Curiosity-driven exploration techniques
    - Example: Curiosity in Montezuma's Revenge
    - Intrinsic motivation and curiosity
    - Curiosity vs. extrinsic rewards
    - Challenges in curiosity-driven exploration
2. Distinction Between Epistemic and Aleatoric Uncertainty
    - Overview of uncertainty in RL
    - Epistemic vs. aleatoric uncertainty
    - Examples and implications
3. Prediction Error as Intrinsic Reward
    - Forward model prediction errors
    - Implementation strategies
    - ICM (Intrinsic Curiosity Module)
4. Surprise and Entropy
    - Information theory foundations
5. Advanced Exploration Algorithms
    - Entropy-regularized on-policy Actor-Critic
    - Soft Actor-Critic (SAC)
6. Summary
7. Outlook

Literature

This lecture draws from the following sources:

Primary Sources:

1. **Eysenbach, B. (2025).** "Lecture 18: Exploration." *COS435/ECE 433: Introduction to Reinforcement Learning*. Princeton University. PDF (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-18-exploration.pdf)
2. **Eysenbach, B. (2025).** "Lecture 19: Exploration." *COS435/ECE 433: Introduction to Reinforcement Learning*. Princeton University. PDF (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)
3. **Levine, S. (2023).** "Lecture 13: Exploration in Deep Reinforcement Learning." *CS285: Deep Reinforcement Learning*. UC Berkeley. PDF (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-13.pdf)
4. **Sutton, R. S., & Barto, A. G. (2018).** *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. Online preprint (https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf)

    - Chapter 15: Intrinsically Motivated Reinforcement Learning (p. 494-497)

Learning Objectives

- Analyze the importance of exploration in RL
- Distinguish between epistemic and aleatoric uncertainty
- Implement intrinsic motivation mechanisms for exploration
- Apply advanced exploration strategies in deep RL

Curiosity and exploration

Overview of curiosity in RL

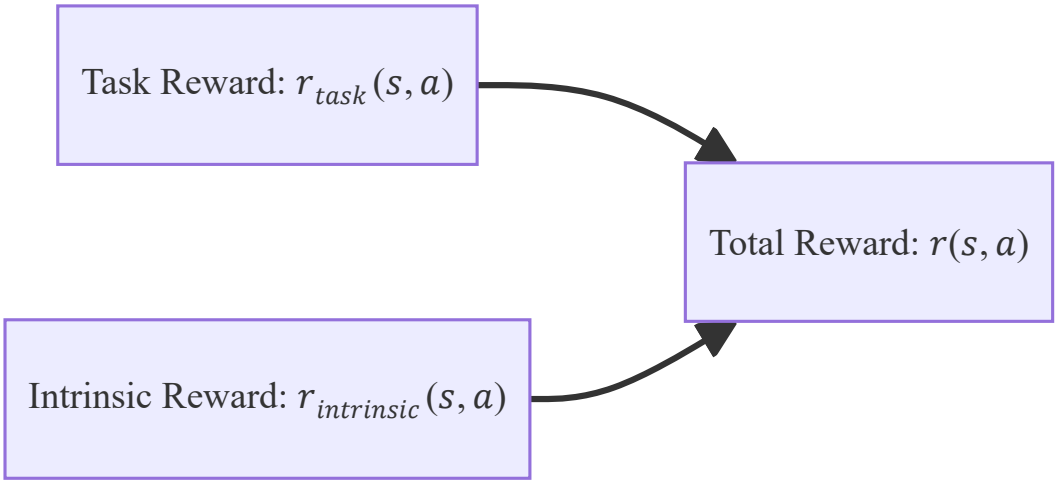
Curiosity as intrinsic motivation: In reinforcement learning, curiosity drives agents to explore their environment beyond external rewards, enabling discovery of novel strategies. (Eysenbach, Lecture 19, 2025, p. 1-2 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf))

- Intrinsic motivation mechanism
- Exploration beyond external rewards
- Discovery of novel strategies

Curiosity enables play and learning: Observations in neuroscience and animal behavior suggest that curiosity-driven play occurs even without extrinsic rewards, highlighting intrinsic motivation's importance in both artificial and biological agents. (Eysenbach, Lecture 19, 2025, p. 1-2 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf))

> **Key insight:** Intrinsic motivation drives play without external rewards

Curiosity-driven exploration techniques



Intrinsic reward formulation: Curiosity-driven exploration augments the RL reward with an intrinsic term:

$$r(s, a) = (1 - \beta) r_{task}(s, a) + \beta r_{intrinsic}(s, a)$$

, where r_{task} is the (normalized) extrinsic task reward, $r_{intrinsic}$ is (normalized) intrinsic reward that encourages exploration and β is a balancing factor.

(Eysenbach, Lecture 19, 2025, p. 1-2 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf))

Types of intrinsic rewards: Common forms include prediction errors in forward/inverse models, uncertainty, entropy, information gain, random network errors, and state visitation counts. (Eysenbach, Lecture 19, 2025, p. 1-2 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf))

Intrinsic Reward Type	Description
Prediction Errors	Forward/Inverse models
Uncertainty / Entropy	Epistemic uncertainty and output entropy
Information Gain	Knowledge acquisition
Random Network Errors	Output discrepancies
State Visitation Counts	Frequency-based exploration

Example: Curiosity in Montezuma's Revenge

Montezuma's Revenge as challenge: The Atari game Montezuma's Revenge is a canonical benchmark for curiosity-driven exploration, featuring sparse rewards and requiring long sequences of unrewarded actions. (Levine, Lecture 13, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-13.pdf))

- Sparse rewards
- Long action sequences required
- Benchmark for exploration

Curiosity-driven methods achieve progress: Curiosity-driven exploration using intrinsic rewards based on novelty or prediction error has enabled agents to outperform traditional exploration strategies in Montezuma's Revenge. (Levine, Lecture 13, 2023 (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-13.pdf))

> **Key result:** Intrinsic rewards enable non-trivial progress

- Outperforms traditional strategies
- Based on novelty/prediction error

In [1]:

```python
# Run DQN agent on Montezuma's Revenge
# This demonstrates standard DQN (without curiosity-driven explor
ation) on a sparse-reward environment

import sys
sys.path.append('.')

# Import and reload the helpers module to ensure latest version i
s used
import importlib
import dqn_helpers
importlib.reload(dqn_helpers)
from dqn_helpers import train_dqn_atari

print("=" * 70)
print("Running DQN on Montezuma's Revenge (2 episodes with render
ing)")
print("=" * 70)
print()
print("Note: This demonstrates the challenge of sparse rewards.")
print("DQN struggles without intrinsic motivation or curiosity.")
print()

# Train/run DQN for 2 episodes with rendering enabled
q_network, episode_rewards = train_dqn_atari(
    env_name='ALE/MontezumaRevenge-v5',
    num_episodes=1,
    max_steps=1000,
    gamma=0.99,
    epsilon_start=1.0,
    epsilon_end=0.1,
    epsilon_decay=0.995,
    learning_rate=1e-4,
    batch_size=32,
    target_update_freq=1000,
    buffer_capacity=10000,
    frame_stack=4,
    render=True  # Enable rendering to visualize the agent
)

print()
print("=" * 70)
print("Results:")
print(f"  Episode 1 Reward: {episode_rewards[0]:.2f}")
print("=" * 70)
print()
print("Observation: Without curiosity-driven exploration, DQN typ
ically")
print("achieves near-zero reward on Montezuma's Revenge due to ex
tremely")
print("sparse reward signals. This motivates the need for intrins
ic rewards!")
```

```
=================================================================
=====
Running DQN on Montezuma's Revenge (2 episodes with rendering)
=================================================================
=====

Note: This demonstrates the challenge of sparse rewards.
DQN struggles without intrinsic motivation or curiosity.

Environment: ALE/MontezumaRevenge-v5
Number of actions: 18
Training for 1 episodes...

Episode 1/1: Reward = 0.00, Epsilon = 0.995, Steps = 349

Training complete!

=================================================================
=====
Results:
  Episode 1 Reward: 0.00
=================================================================
=====

Observation: Without curiosity-driven exploration, DQN typically
achieves near-zero reward on Montezuma's Revenge due to extremely
sparse reward signals. This motivates the need for intrinsic rewa
rds!
```

Improving exploration

- In Montezumas Revenge (and other applications), an agent must explore in a meaningful way to avoid local optima.
- Exploration can be seen as finding states about which the agent is uncertain.
- Approach: Learn an exploration policy that maximizes uncertainty.
- But what is uncertainty?
  - When our predictions are wrong?
  - When we don't know what to do?
  - Uncertainty in the data?
  - Uncertainty in the environment itself?

Prediction error as intrinsic reward

Intrinsic reward from prediction error: In RL, intrinsic rewards can be generated from prediction errors in a predictive model, incentivizing agents to explore states where their predictions are inaccurate or uncertain.

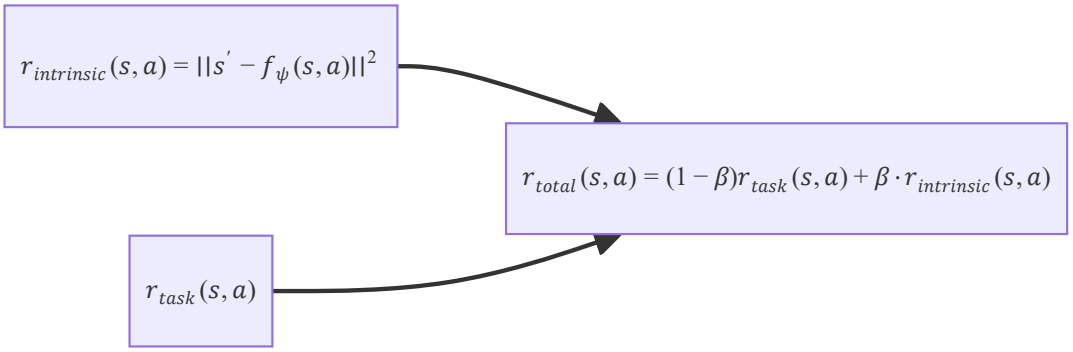
- Intrinsic rewards from prediction errors
- Encourages exploration of uncertain states
- Incentivizes learning in RL

Encouraging exploration via model error: By rewarding high prediction error, RL agents are encouraged to visit novel or less-understood states, thus facilitating exploration and the discovery of new strategies beyond those driven by extrinsic rewards.

> **Exploration via Model Error:**
>
> - Rewards high prediction error
> - Encourages novel state visitation
> - Facilitates new strategy discovery

[(COS435/ ECE 433: Introduction to RL Lecture 19: Exploration, Ben Eysenbach, 2025, p. 1-2, search for "Errors in a predictive model") (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)

Balancing prediction error with task-specific rewards



[(COS435/ ECE 433: Introduction to RL Lecture 19: Exploration, Ben Eysenbach, 2025, p. 1-2, search for "r(s, a)=r_{task}(s, a)+r_{intrinsic}(s, a)") (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)

Key components:

- **Forward model** $f_\psi(s,a)$: Predicts next state from current state and action
- **Prediction error**: $r_{intrinsic}(s,a) = \text{normalize}(||s' - f_\psi(s,a)||^2)$ High in novel/uncertain states → encourages exploration
- **Balance factor** β: Controls trade-off between exploration and exploitation

Tuning intrinsic reward weight: The effectiveness depends on appropriately tuning β:

- Too high: Agent focuses on prediction errors, ignores task objectives
- Too low: Insufficient exploration bonus
- Typical range: $\beta \in [0.01, 1.0]$

[(COS435/ ECE 433: Introduction to RL Lecture 19: Exploration, Ben Eysenbach, 2025, p. 1-2) (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)

Algorithm: DDPG with Forward Model Intrinsic Reward (simplified, without target networks)

$$
\begin{aligned}
&\textbf{Input:} \text{ Environment Env, intrinsic reward weight } \beta, \text{ discount factor } \gamma \\
&\textbf{Output:} \text{ Trained actor } \pi_\theta \text{ and forward model } f_\psi \\
&\\
&\textbf{1.} \text{ Initialize } \pi_\theta(s), Q_\phi(s,a), f_\psi(s,a), \text{ replay buffer } \mathcal{D} \leftarrow \emptyset \\
&\textbf{2. For } \text{episode} = 1 \text{ to } N\textbf{:} \\
&\quad \textbf{3.} \; s \leftarrow \text{Env.reset}() \\
&\quad \textbf{4. For } t = 1 \text{ to } T\textbf{:} \\
&\quad\quad \textbf{5.} \; a \leftarrow \pi_\theta(s) + \text{noise} \\
&\quad\quad \textbf{6.} \; s', r_{\text{task}}, \text{done} \leftarrow \text{Env.step}(a) \\
&\quad\quad \textbf{7.} \; \mathcal{D}.\text{store}(s, a, r_{\text{task}}, s', \text{done}) \\
&\quad\quad \textbf{8. If } |\mathcal{D}| \geq \text{batch_size}\textbf{:} \\
&\quad\quad\quad \textbf{9.} \text{ Sample batch } \mathcal{B} = \{(s_i, a_i, \text{task}, i, s'_i, \text{done}_i)\}_{i=1}^{B} \sim \mathcal{D} \\
&\quad\quad\quad \textbf{10.} \; r_{\text{intrinsic},i} \leftarrow \text{normalize}(||s'_i - f_\psi(s_i, a_i)||^2) \text{ for each } i \in \mathcal{B} \\
&\quad\quad\quad \textbf{11.} \text{ Update } f_\psi \text{ to minimize } ||s'_i - f_\psi(s_i, a_i)||^2 \\
&\quad\quad\quad \textbf{12.} \; y_i \leftarrow (1-\beta) \cdot r_{\text{task},i} + \beta \cdot r_{\text{intrinsic},i} + \gamma(1 - \text{done}_i) \cdot Q_\phi(s'_i, \pi_\theta(s'_i)) \\
&\quad\quad\quad \textbf{13.} \text{ Update } Q_\phi \text{ to minimize } (Q_\phi(s_i, a_i) - y_i)^2 \\
&\quad\quad\quad \textbf{14.} \text{ Update } \pi_\theta \text{ to maximize } Q_\phi(s_i, \pi_\theta(s_i)) \\
&\quad\quad \textbf{15.} \; s \leftarrow s' \\
&\quad\quad \textbf{16. If } \text{done}\textbf{: break} \\
&\textbf{17. Return } \pi_\theta, f_\psi
\end{aligned}
$$

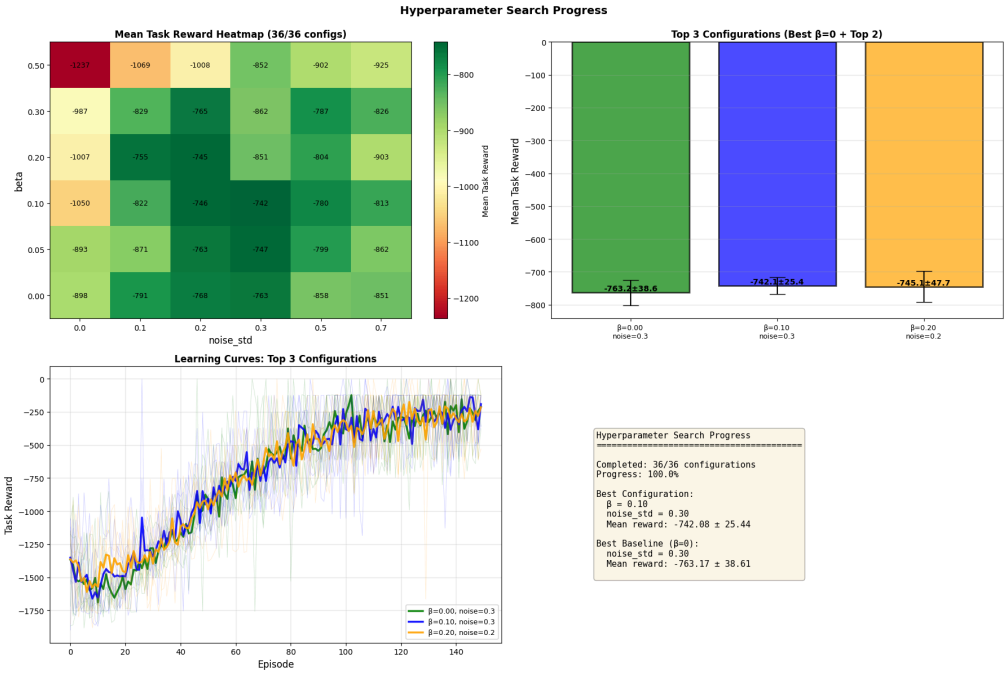
Differences from original DDPG:

- Forward model trained alongside actor-critic
- Prediction error as intrinsic reward encourages exploration in states with high prediction error
- No target networks (θ', ϕ') for stability (simplified version)

In [1]:
```
# Hyperparameter search: Find optimal noise_std and beta
import pandas as pd
from IPython.display import clear_output
import os

# Reload module for development
import importlib
import ddpg_intrinsic_helpers
importlib.reload(ddpg_intrinsic_helpers)
from ddpg_intrinsic_helpers import (train_ddpg_pred_err,
                                    run_config_trials, plot_sear
ch_progress)

# Setup hyperparameter grid
beta_values = [0.0, 0.05, 0.1, 0.2, 0.3, 0.5]
noise_std_values = [0.0, 0.1, 0.2, 0.3, 0.5, 0.7]
num_trials = 7  # Run each configuration 3 times
num_episodes = 150  # Shorter runs for hyperparameter search

all_params = [{'beta': b, 'noise_std': ns} for b in beta_values f
or ns in noise_std_values]

cache_dir = 'hyperparam_search/pendulum'  # Directory for caching
trial results

print(f"Running hyperparameter search:")
print(f"  - {len(all_params)} configs ({len(beta_values)} beta ×
{len(noise_std_values)} noise_std)")
print(f"  - {num_trials} trials per config")
print(f"  - {len(all_params) * num_trials} total runs")
print(f"  - {num_episodes} episodes per run")
print(f"  - Cache directory: {cache_dir}")
print(f"  - Using cached trials when available (resumable search)
\n")

# Run experiments with live plotting
results = []

for idx, params in enumerate(all_params):
    beta = params['beta']
    noise_std = params['noise_std']

    # Run trials for this configuration (uses caching)
    config_result = run_config_trials(
        train_ddpg_pred_err, beta, noise_std, num_trials, num_epi
sodes,
        cache_dir=cache_dir,
        num_workers=5
    )
    results.append(config_result)

    # Update plots after each parameter configuration
    clear_output(wait=True)

    # Convert current results to DataFrame
    df_results = pd.DataFrame(results)
```

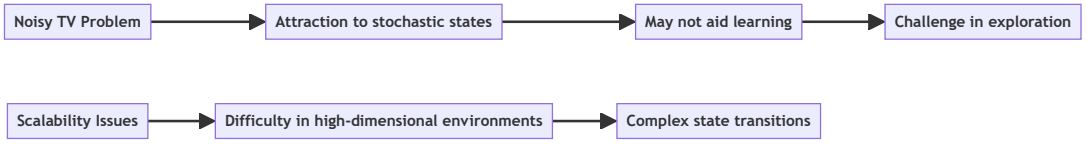
```
    # Find best overall configuration so far
    best_overall = df_results.loc[df_results['mean_reward'].idxma
x()]

    print(f"\nProgress: {idx+1}/{len(all_params)} configurations
completed")
    print(f"Current config: β={beta:.2f}, noise_std={noise_std:.2
f}")
    print(f"Current result: {config_result['mean_reward']:.2f} ±
{config_result['std_reward']:.2f}")
    print(f"\nBest so far:")
    print(f"  β={best_overall['beta']:.2f}, noise_std={best_overa
ll['noise_std']:.2f}")
    print(f"  Mean reward: {best_overall['mean_reward']:.2f} ± {b
est_overall['std_reward']:.2f}")
    print("="*70)

    # Visualize current results
    plot_search_progress(df_results, idx, len(all_params))

# Final summary
df_results = pd.DataFrame(results)
best_overall = df_results.loc[df_results['mean_reward'].idxmax()]

print("\n" + "="*70)
print("HYPERPARAMETER SEARCH COMPLETE")
print("="*70)
print(f"\nBest configuration:")
print(f"  beta = {best_overall['beta']:.2f}")
print(f"  noise_std = {best_overall['noise_std']:.2f}")
print(f"  Mean reward = {best_overall['mean_reward']:.2f} ± {best
_overall['std_reward']:.2f}")

# Also show best beta=0 for comparison
best_beta_zero = df_results[df_results['beta'] == 0.0].loc[
    df_results[df_results['beta'] == 0.0]['mean_reward'].idxmax
()]

print(f"\nBest β=0 configuration (baseline):")
print(f"  noise_std = {best_beta_zero['noise_std']:.2f}")
print(f"  Mean reward = {best_beta_zero['mean_reward']:.2f} ± {be
st_beta_zero['std_reward']:.2f}")

if best_overall['beta'] > 0.0:
    improvement = best_overall['mean_reward'] - best_beta_zero['m
ean_reward']
    improvement_pct = (improvement / abs(best_beta_zero['mean_rew
ard'])) * 100
    print(f"\nImprovement over best baseline: {improvement:.2f}
({improvement_pct:.1f}%)")
else:
    print(f"\nNote: Best overall configuration has β=0.0 (baselin
e)")

print("="*70)
```

```
Progress: 36/36 configurations completed
Current config: β=0.50, noise_std=0.70
Current result: -924.56 ± 104.78

Best so far:
  β=0.10, noise_std=0.30
  Mean reward: -742.08 ± 25.44
================================================================
=====
```




```
================================================================
=====
HYPERPARAMETER SEARCH COMPLETE
================================================================
=====

Best configuration:
  beta = 0.10
  noise_std = 0.30
  Mean reward = -742.08 ± 25.44

Best β=0 configuration (baseline):
  noise_std = 0.30
  Mean reward = -763.17 ± 38.61

Improvement over best baseline: 21.09 (2.8%)
================================================================
=====
```

Challenges in using prediction error for exploration



Noisy TV problem: When using prediction error as intrinsic reward, agents can get stuck exploring inherently stochastic (noisy) states that provide no useful information. This is analogous to watching a TV with random static—high prediction error, but no learning value.

[(COS435/ ECE 433: Introduction to RL Lecture 19: Exploration, Ben Eysenbach, 2025, p. 1-2, search for "noisy TV problem, where transitions are highly stochastic") (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)

- Attraction to stochastic states
- May not aid learning
- Challenge in exploration

Scalability to high-dimensional settings: Prediction error-based exploration methods can struggle to scale effectively in high-dimensional environments, as accurately modeling and predicting complex state transitions becomes increasingly difficult.

[(COS435/ ECE 433: Introduction to RL Lecture 19: Exploration, Ben Eysenbach, 2025, p. 1-2, search for "how well does the method scale to high-dimensional settings") (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)

- Difficulty in high-dimensional environments
- Complex state transitions
- Scalability issues

Noisy Pendulum Environment

Let's create a version of Pendulum with artificial noise that increases when the pendulum is hanging down (angle-dependent noise).

Algorithm: Noisy Pendulum Wrapper (Pseudocode)

$$
\begin{aligned}
&\textbf{Input:} \text{ Base environment Env, noise scale } \sigma_{\text{scale}} \\
&\textbf{Output:} \text{ Wrapped environment with angle-dependent noise} \\
&\\
&\textbf{Class NoisyPendulum(Wrapper):} \\
&\quad \textbf{1.} \text{ Initialize: env} \leftarrow \text{Env, noise_scale} \leftarrow \sigma_{\text{scale}} \\
&\\
&\quad \textbf{2. Function reset():} \\
&\qquad \textbf{3.} \; s \leftarrow \text{env.reset}() \\
&\qquad \textbf{4.} \; s_{\text{noisy}} \leftarrow \text{add_noise}(s) \\
&\qquad \textbf{5. Return } s_{\text{noisy}} \\
&\\
&\quad \textbf{6. Function step}(a): \\
&\qquad \textbf{7.} \; s', r \leftarrow \text{env.step}(a) \\
&\qquad \textbf{8.} \; s'_{\text{noisy}} \leftarrow \text{add_noise}(s') \\
&\qquad \textbf{9. Return } s'_{\text{noisy}}, r \\
&\\
&\quad \textbf{10. Function add_noise}(s): \\
&\qquad \textbf{11.} \; \cos\theta, \sin\theta, \dot{\theta} \leftarrow s \quad \text{(parse state)} \\
&\qquad \textbf{12.} \; \sigma_{\text{noise}} \leftarrow \sigma_{\text{scale}} \cdot \frac{1+\cos\theta}{2} \quad \text{(angle-dependent noise)} \\
&\qquad \textbf{13.} \; \epsilon \sim \mathcal{N}(0, \sigma^2_{\text{noise}} I_3) \quad \text{(sample Gaussian noise)} \\
&\qquad \textbf{14.} \; s_{\text{noisy}} \leftarrow \text{clip}(s+\epsilon, s_{\min}, s_{\max}) \quad \text{(add noise and clip)} \\
&\qquad \textbf{15. Return } s_{\text{noisy}}
\end{aligned}
$$

Key features:

- **Line 12**: Noise standard deviation proportional to how much pendulum hangs down (cos θ ≈ 1 → high noise, cos θ ≈ -1 → low noise)
- **Line 13**: Independent Gaussian noise for each observation dimension
- **Line 14**: Clip to valid observation space bounds
- **Effect**: Hanging-down position becomes "noisy TV" region with high aleatoric uncertainty

In [2]:

```python
# Implementation of NoisyPendulum wrapper
import gymnasium as gym
import numpy as np

class NoisyPendulum(gym.Wrapper):
    """
    Pendulum with position-dependent observation noise (noisy TV
problem).

    The closer the pendulum is to hanging down (starting positio
n),
    the more noise is added to observations. This creates regions
of
    high aleatoric uncertainty that should NOT be rewarded for ex
ploration.

    Args:
        env: Base Pendulum environment
        noise_scale: Scaling factor for position-dependent noise
    """
    def __init__(self, env, noise_scale=0.5):
        super().__init__(env)
        self.noise_scale = noise_scale

    def reset(self, **kwargs):
        """Reset environment and add noise to initial observatio
n."""
        obs, info = self.env.reset(**kwargs)
        return self._add_noise(obs), info

    def step(self, action):
        """Execute action and add noise to resulting observatio
n."""
        obs, reward, terminated, truncated, info = self.env.step
(action)
        return self._add_noise(obs), reward, terminated, truncate
d, info

    def _add_noise(self, obs):
        """Add noise proportional to how much the pendulum is han
ging down."""
        cos_theta, sin_theta, theta_dot = obs

        # Noise increases when pendulum is hanging down (cos_thet
a ≈ 1)
        # Noise decreases when pendulum is upright (cos_theta ≈ -
1)
        # Mapping: cos_theta ∈ [-1, 1] → noise_factor ∈ [0, 1]
        noise_factor = (1 + cos_theta) / 2
        noise_std = self.noise_scale * noise_factor

        # Add Gaussian noise to observations
        noise = np.random.randn(3) * noise_std
        noisy_obs = obs + noise

        # Clip to valid observation space
```

```
        noisy_obs = np.clip(noisy_obs,
                            self.observation_space.low,
                            self.observation_space.high)

        return noisy_obs

    def _add_noise_velocity(self, obs):
        """Add noise proportional to angular velocity (old implem
entation)."""
        cos_theta, sin_theta, theta_dot = obs

        # Noise increases with |angular velocity|
        velocity_magnitude = np.abs(theta_dot)
        noise_std = self.noise_scale * velocity_magnitude

        # Add Gaussian noise to observations
        noise = np.random.randn(3) * noise_std
        noisy_obs = obs + noise

        # Clip to valid observation space
        noisy_obs = np.clip(noisy_obs,
                            self.observation_space.low,
                            self.observation_space.high)

        return noisy_obs

# Test the noisy environment
base_env = gym.make('Pendulum-v1')
noisy_env = NoisyPendulum(base_env, noise_scale=0.5)

print("NoisyPendulum environment created successfully!")
print(f"Observation space: {noisy_env.observation_space}")
print(f"Action space: {noisy_env.action_space}")

# Test a few steps
obs, _ = noisy_env.reset()
cos_theta = obs[0]
noise_factor = (1 + cos_theta) / 2
print(f"\nInitial observation (with noise): {obs}")
print(f"cos(theta)={cos_theta:.2f}, noise_factor={noise_factor:.2
f}")

for i in range(3):
    action = noisy_env.action_space.sample()
    obs, reward, terminated, truncated, _ = noisy_env.step(actio
n)
    cos_theta = obs[0]
    noise_factor = (1 + cos_theta) / 2
    print(f"Step {i+1}: cos(theta)={cos_theta:.2f}, noise_factor=
{noise_factor:.2f}, obs={obs}")

noisy_env.close()
base_env.close()
```

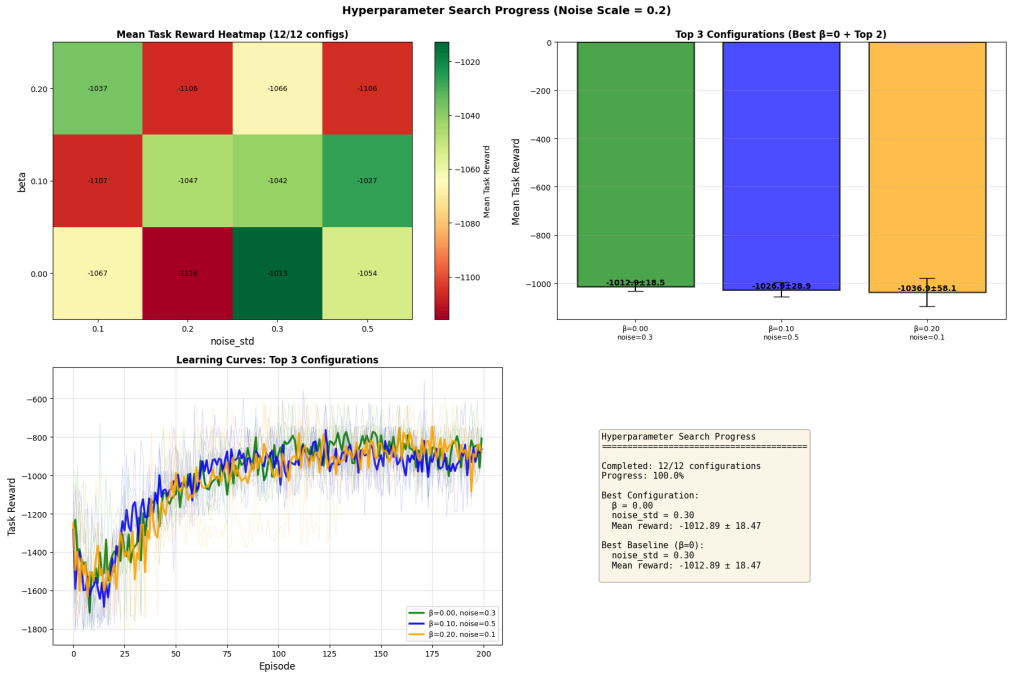
```
NoisyPendulum environment created successfully!
Observation space: Box([-1. -1. -8.], [1. 1. 8.], (3,), float32)
Action space: Box(-2.0, 2.0, (1,), float32)

Initial observation (with noise): [ 0.90699478  0.64586382 -0.748
66065]
cos(theta)=0.91, noise_factor=0.95
Step 1: cos(theta)=0.30, noise_factor=0.65, obs=[0.30249078 0.132
58868 0.72780152]
Step 2: cos(theta)=0.38, noise_factor=0.69, obs=[0.3763744  0.418
45133 0.90316566]
Step 3: cos(theta)=0.66, noise_factor=0.83, obs=[0.66413191 0.601
93942 1.5588963 ]
```

In [3]:

```python
# Hyperparameter search for NoisyPendulum with different noise_sc
ale values
import pandas as pd
from IPython.display import clear_output, display
import os
import importlib
import ddpg_intrinsic_helpers
importlib.reload(ddpg_intrinsic_helpers)
from ddpg_intrinsic_helpers import (train_ddpg_pred_err,
                                    run_config_trials, plot_sear
ch_progress)
from gymnasium.envs.registration import register

# Setup hyperparameter grid
beta_values = [0.0, 0.1, 0.2]
noise_std_values = [0.1, 0.2, 0.3, 0.5]
num_trials = 5  # Run each configuration 7 times
num_episodes = 200  # Episodes per run

all_params = [{'beta': b, 'noise_std': ns} for b in beta_values f
or ns in noise_std_values]
```

In [4]:
```
import gymnasium as gym
import importlib
import ddpg_intrinsic_helpers
importlib.reload(ddpg_intrinsic_helpers)
from ddpg_intrinsic_helpers import (train_ddpg_pred_err,
                                    run_config_trials, plot_sear
ch_progress)

# Import the noisy pendulum environment module (picklable for mul
tiprocessing)
import noisy_pendulum_env
importlib.reload(noisy_pendulum_env)
from noisy_pendulum_env import register_noisy_pendulum_env
from functools import partial

# SET NOISE SCALE HERE
noise_scale = 0.2

print("\n" + "="*70)
print(f"HYPERPARAMETER SEARCH FOR NOISE_SCALE = {noise_scale}")
print("="*70)

# Register NoisyPendulum in main process
register_noisy_pendulum_env(noise_scale=noise_scale)

# Create a partial function that captures noise_scale for worker
processes
# partial() objects are picklable
register_env_worker = partial(register_noisy_pendulum_env, noise_
scale=noise_scale)

cache_dir = f'hyperparam_search/noisy_pendulum_scale_{noise_scal
e}'

# Run experiments with live plotting and multiprocessing
results = []

for idx, params in enumerate(all_params):
    beta = params['beta']
    noise_std = params['noise_std']

    # Run trials for this configuration (uses caching and multipr
ocessing)
    config_result = run_config_trials(
        train_ddpg_pred_err, beta, noise_std, num_trials, num_epi
sodes,
        cache_dir=cache_dir, env_name='NoisyPendulum-v0',
        num_workers=5,  # Use 5 parallel workers
        env_registration_fn=register_env_worker  # Pass picklable
partial function
    )
    results.append(config_result)

    clear_output(wait=True)

    # Convert current results to DataFrame
```

```
df_results = pd.DataFrame(results)

plot_search_progress(df_results, idx, len(all_params),
                     noise_scale=noise_scale)
```



Uncertainty

Epistemic vs. aleatoric uncertainty

Uncertainty Type	Nature	Source	Also Known As	Examples
Epistemic	Reducible with more data	Limited knowledge	Model uncertainty	Playing a game we have not played before Visting places we have not seen before
Aleatoric	Irreducible randomness	Inherent randomness	Statistical uncertainty	Coin toss outcomes Noisy TV screen

([Eysenbach, Lecture 19, 2025, p. 2-3 (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf))

Estimating epistemic uncertainty with ensembles or Bayesian neural networks

---

Ensemble-based epistemic uncertainty (used in this lecture)

Ensembles for epistemic uncertainty: Ensembles of Q-networks or models are commonly used in RL to estimate epistemic uncertainty, as disagreement among ensemble members indicates areas where the agent lacks sufficient knowledge.

[(COS435/ ECE 433: Introduction to RL Lecture 19: Exploration, Ben Eysenbach, 2025, p. 2-3, search for "ensembles give us a convenient way") (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-19-exploration.pdf)

Given an ensemble of K neural networks $\{f_1, f_2, \ldots, f_K\}$, epistemic uncertainty of the output for a value x is quantified by the **disagreement** among ensemble predictions:

$$\text{Epistemic}(x) = \text{Var}_k[f_k(x)]$$

In the context of RL, the value x might be a state-action pair, and the output uncertainty might be the uncertainty of a Q-function or a forward model or a policy.

Key properties:

- High variance indicates epistemic uncertainty (models disagree)
- Low variance indicates confidence (models agree)
- **This approach will be used throughout this lecture**

Bayesian neural networks (alternative, not covered here)

Bayesian neural networks estimate epistemic uncertainty by maintaining a distribution over model parameters, allowing the agent to quantify its uncertainty about predictions due to limited data.

[(CS285 Lecture 11: Model-Based Reinforcement Learning, Sergey Levine, 2018, p. 18-33, search for "Bayesian neural networks") (https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-11.pdf)](https://rail.eecs.berkeley.edu/deeprlcourse-fa23/deeprlcourse-fa23/static/slides/lec-11.pdf)

- Distribution over parameters $p(\theta \mid \mathcal{D})$
- Predictive uncertainty via marginalization
- **Not covered in this lecture** (ensembles are more practical)

Estimating aleatoric uncertainty via Maximum Likelihood Estimation (MLE)

A standard and principled way to model **aleatoric uncertainty in regression** is to let the neural network predict a *full conditional distribution* over outputs rather than a single point estimate.

A common assumption is a Gaussian observation model:

$$p(y \mid x) = \mathcal{N}\big(y \mid \mu(x), \sigma^2(x)\big)$$

Concretely, the network takes input x and predicts both the **mean** and **variance** of the output distribution over y:

$$\mu(x) = \mathbb{E}[y \mid x], \quad \sigma^2(x) = \mathrm{Var}[y \mid x]$$

where $\mu(x)$ and $\sigma^2(x)$ are **functions of the input** x that parameterize the predictive distribution over the output y.

Training is performed by **maximum likelihood estimation**, i.e. minimizing the negative log-likelihood (NLL): TODO: Explain / rehearse NLL and likelihood here.

$$\mathcal{L}_{\mathrm{NLL}}(x, y) = -\log p(y \mid x) = \frac{1}{2}\left(\frac{(y - \mu(x))^2}{\sigma^2(x)} + \log \sigma^2(x)\right)$$

Key properties:

- Large prediction errors increase the first term
- Inflating the variance is penalized by the $\log \sigma^2(x)$ term

In practice, the network predicts $\log \sigma^2(x)$ to ensure numerical stability:

$$\sigma^2(x) = \exp(s(x))$$

After training, the predicted variance $\sigma^2(x)$ directly represents **aleatoric uncertainty**, i.e. noise inherent in the data-generating process.

- High $\sigma^2(x)$: inherently noisy or ambiguous targets
- Low $\sigma^2(x)$: reliable, low-noise predictions

This uncertainty is **irreducible**, even with infinite data.

Combining ensembles and MLE: Epistemic and aleatoric uncertainty together

Both approaches can be **combined** by training an **ensemble of networks**, where each network predicts both mean and variance:

$$\text{Ensemble member } k: \quad \mu_k(x),\ \sigma_k^2(x)$$

This hybrid approach enables simultaneous estimation of both epistemic and aleatoric uncertainty. (e.g. Liu et al., 2021 (https://www.sciencedirect.com/science/article/abs/pii/S1270963821006489))

Mathematical formulation:

- **Aleatoric uncertainty:** Mean of predicted variances across the ensemble

$$\text{Aleatoric}(x) = \frac{1}{K}\sum_{k=1}^{K}\sigma_k^2(x)$$

- **Epistemic uncertainty:** Variance of predicted means across the ensemble

$$\text{Epistemic}(x) = \text{Var}_k[\mu_k(x)] = \frac{1}{K}\sum_{k=1}^{K}\left(\mu_k(x) - \bar{\mu}(x)\right)^2$$

- **Epistemic uncertainty of aleatoric uncertainty:** Variance of predicted variances

$$\text{Epistemic}_{\text{aleatoric}}(x) = \text{Var}_k[\sigma_k^2(x)]$$

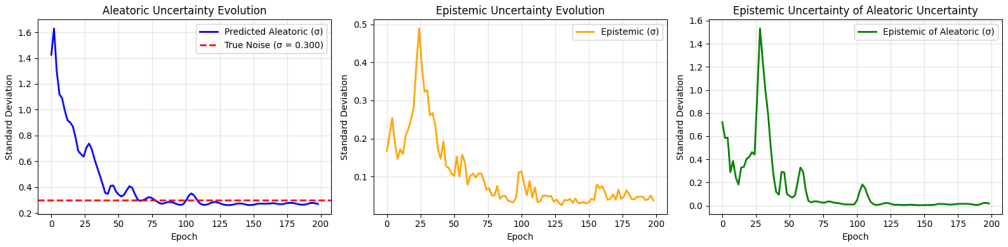
This quantifies the uncertainty about the uncertainty itself (disagreement on noise levels).

In [5]:
```
# Hybrid Ensemble Model: Combining Ensembles and MLE for Epistemi
c and Aleatoric Uncertainty
# Each ensemble member predicts both mean and variance

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# Import helper functions (reload to get latest version)
import importlib
import helper_functions
importlib.reload(helper_functions)
from helper_functions import plot_uncertainty_evolution

class EnsembleMember(nn.Module):
    """Single ensemble member that predicts both mean and varianc
e (log-variance for stability)"""
    def __init__(self, input_dim=1, hidden_dim=64):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
        )
        self.mean_head = nn.Linear(hidden_dim, 1)
        self.log_var_head = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        features = self.network(x)
        mean = self.mean_head(features)
        log_var = self.log_var_head(features)
        return mean, log_var

class HybridEnsemble:
    """Ensemble of networks for epistemic and aleatoric uncertain
ty estimation"""
    def __init__(self, n_members=5, input_dim=1, hidden_dim=64, l
r=0.01):
        self.n_members = n_members
        self.members = [EnsembleMember(input_dim, hidden_dim) for
_ in range(n_members)]
        self.optimizers = [optim.Adam(m.parameters(), lr=lr) for
m in self.members]

    def nll_loss(self, mean, log_var, target):
        """Negative log-likelihood loss for Gaussian with predict
ed mean and variance"""
        var = torch.exp(log_var)
        loss = 0.5 * ((target - mean)**2 / var + log_var)
        return loss.mean()

    def train_step(self, x, y):
        """Train all ensemble members on a batch"""
        losses = []
```

```
        for member, optimizer in zip(self.members, self.optimizer
s):
            optimizer.zero_grad()
            mean, log_var = member(x)
            loss = self.nll_loss(mean, log_var, y)
            loss.backward()
            optimizer.step()
            losses.append(loss.item())
        return np.mean(losses)

    def predict(self, x):
        """Get predictions from all ensemble members"""
        means, vars = [], []
        with torch.no_grad():
            for member in self.members:
                mean, log_var = member(x)
                means.append(mean.numpy())
                vars.append(np.exp(log_var.numpy()))
        return np.array(means), np.array(vars)

    def compute_uncertainties(self, x):
        """Compute all three types of uncertainty"""
        means, vars = self.predict(x)

        # Aleatoric: mean of predicted variances
        aleatoric = np.mean(vars, axis=0)

        # Epistemic: variance of predicted means
        epistemic = np.var(means, axis=0)

        # Epistemic of aleatoric: variance of predicted variances
        epistemic_aleatoric = np.var(vars, axis=0)

        return aleatoric, epistemic, epistemic_aleatoric

# Generate artificial data with constant noise (homoscedastic)
np.random.seed(42)
torch.manual_seed(42)

# Input space
x_train = np.linspace(-4, 4, 200).reshape(-1, 1)

# True function: nonlinear with CONSTANT noise
true_mean = np.sin(x_train) * x_train
true_noise_std = 0.3  # Constant noise standard deviation
y_train = true_mean + np.random.randn(*x_train.shape) * true_nois
e_std

# Test points
x_test = np.linspace(-4, 4, 100).reshape(-1, 1)

# Convert to tensors
x_train_t = torch.FloatTensor(x_train)
y_train_t = torch.FloatTensor(y_train)
x_test_t = torch.FloatTensor(x_test)

# Initialize ensemble
```

```python
ensemble = HybridEnsemble(n_members=5, hidden_dim=64, lr=0.01)

# Training loop with uncertainty tracking
n_epochs = 200
track_every = 2
uncertainties_over_time = []

print("Training hybrid ensemble model with constant noise...")
print(f"True noise standard deviation: {true_noise_std:.3f}\n")

for epoch in range(n_epochs):
    loss = ensemble.train_step(x_train_t, y_train_t)

    if epoch % track_every == 0:
        aleatoric, epistemic, epistemic_aleatoric = ensemble.comp
ute_uncertainties(x_test_t)
        uncertainties_over_time.append({
            'epoch': epoch,
            'aleatoric': aleatoric.copy(),
            'epistemic': epistemic.copy(),
            'epistemic_aleatoric': epistemic_aleatoric.copy()
        })
        if epoch % 100 == 0:
            print(f"Epoch {epoch:4d} | Loss: {loss:.4f}")

print("\nTraining complete!")

# Plot uncertainty evolution over epochs
plot_uncertainty_evolution(uncertainties_over_time, true_noise_st
d=true_noise_std)

print("\nKey Observations:")
print(f"  - Aleatoric uncertainty converges to the true noise lev
el (σ = {true_noise_std:.3f})")
print(f"  - Epistemic uncertainty decreases as ensemble learns th
e function")
print(f"  - Epistemic of aleatoric decreases as ensemble agrees o
n noise level")
```

```
Training hybrid ensemble model with constant noise...
True noise standard deviation: 0.300

Epoch    0 | Loss: 1.0058
Epoch  100 | Loss: -0.7981

Training complete!
```




```
Final Uncertainty Values (mean over test points):
  Aleatoric (σ):              0.2710
  True Noise (σ):             0.3000
  Error:                      0.0290
  Epistemic (σ):              0.0376
  Epistemic of Aleatoric (σ): 0.0181

Key Observations:
  - Aleatoric uncertainty converges to the true noise level (σ =
0.300)
  - Epistemic uncertainty decreases as ensemble learns the functi
on
  - Epistemic of aleatoric decreases as ensemble agrees on noise
level
```

Noisy TV Problem and Epistemic Uncertainty

Reconsider not the Noisy TV problem:

Solution: Distinguish Epistemic from Aleatoric Uncertainty

- **Aleatoric uncertainty**: Irreducible randomness in the environment (e.g., stochastic transitions, or noise on a TV)
- **Epistemic uncertainty**: Reducible uncertainty due to lack of knowledge about the dynamics

Key idea: Reward epistemic uncertainty (which decreases with learning) but not aleatoric uncertainty (which remains constant).

DDPG with Epistemic Uncertainty Intrinsic Reward

To solve the noisy TV problem, we use an **ensemble of forward models** to estimate:

1. **Epistemic uncertainty**: Disagreement between ensemble members (variance of predictions)
2. **Aleatoric uncertainty**: Average predicted variance across ensemble

Intrinsic reward based on epistemic uncertainty only:

$$r_{\text{intrinsic}}(s, a) = \text{Var}_k[f_{\psi_k}(s, a)] = \frac{1}{K} \sum_{k=1}^{K} ||f_{\psi_k}(s, a) - \bar{f}(s, a)||^2$$

where $\bar{f}(s, a) = \frac{1}{K} \sum_{k=1}^{K} f_{\psi_k}(s, a)$ is the mean prediction across ensemble.

Key insight: Epistemic uncertainty decreases as the ensemble learns, so it naturally decays in familiar states. Aleatoric uncertainty remains constant regardless of experience.

Algorithm: DDPG with Epistemic Uncertainty Intrinsic Reward (simplified, without target networks)

Input: Environment Env, intrinsic reward weight β, discount factor γ, ensemble size
Output: Trained actor π_θ and forward model ensemble $\{f_{\psi_k}\}_{k=1}^K$

1. Initialize $\pi_\theta(s), Q_\phi(s,a), \{f_{\psi_k}(s,a)\}_{k=1}^K$, replay buffer $\mathcal{D} \leftarrow \emptyset$
2. For episode $= 1$ to N**:**
  3. $s \leftarrow \text{Env.reset}()$
  4. For $t = 1$ to T**:**
    5. $a \leftarrow \pi_\theta(s) + \text{noise}$
    6. $s', r_{\text{task}}, \text{done} \leftarrow \text{Env.step}(a)$
    7. $\mathcal{D}.\text{store}(s, a, r_{\text{task}}, s', \text{done})$
    8. If $|\mathcal{D}| \geq \text{batch_size}$**:**
      9. Sample batch $\mathcal{B} = \{(s_i, a_i, r_{\text{task},i}, s'_i, \text{done}_i)\}_{i=1}^B \sim \mathcal{D}$
      10. Compute epistemic uncertainty for each transition:
        $\hat{s}'_{k,i} \leftarrow f_{\psi_k}(s_i, a_i)$ for each $k \in [1, K]$
        $$\bar{s}'_i \leftarrow \frac{1}{K}\sum_{k=1}^K \hat{s}'_{k,i} \quad \text{(ensemble mean)}$$
        $$r_{\text{intrinsic},i} \leftarrow \text{normalize}\left(\frac{1}{K}\sum_{k=1}^K ||\hat{s}'_{k,i} - \bar{s}'_i||^2\right) \text{ for each } i \in \mathcal{B}$$
      11. Update each f_{ψ_k} to minimize $||s'_i - f_{\psi_k}(s_i, a_i)||^2$
      12. $y_i \leftarrow (1-\beta) \cdot r_{\text{task},i} + \beta \cdot r_{\text{intrinsic},i} + \gamma(1 - \text{done}_i) \cdot Q_\phi(s'_i, \pi_\theta(s'_i))$
      13. Update Q_ϕ to minimize $(Q_\phi(s_i, a_i) - y_i)^2$
      14. Update π_θ to maximize $Q_\phi(s_i, \pi_\theta(s_i))$
    15. $s \leftarrow s'$
    16. If done**: break**
17. Return $\pi_\theta, \{f_{\psi_k}\}_{k=1}^K$

Differences from prediction error approach:

- **Line 10**: Compute epistemic uncertainty as ensemble disagreement (variance across K models) instead of single model prediction error
- **Line 11**: Update K forward models instead of just 1
- **Benefit**: Avoids noisy TV problem by measuring epistemic (reducible) uncertainty, ignoring aleatoric (irreducible) uncertainty
- **Cost**: Requires training K forward models instead of 1 (typically $K = 5 - 10$)

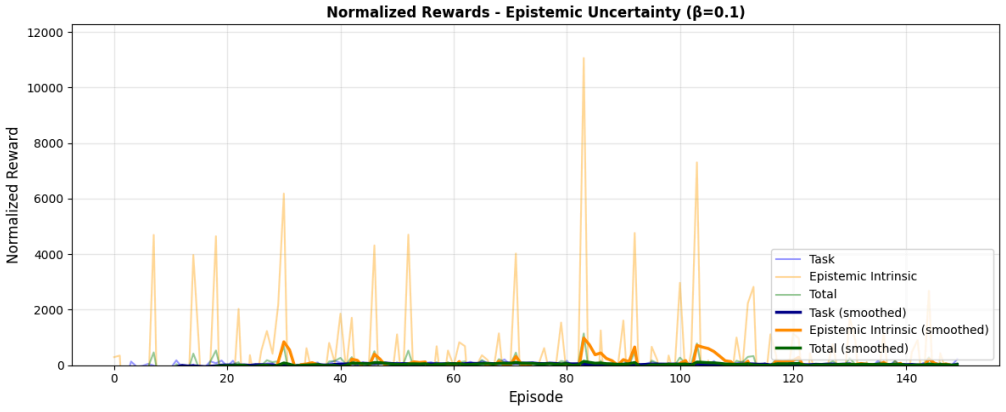
In [6]:

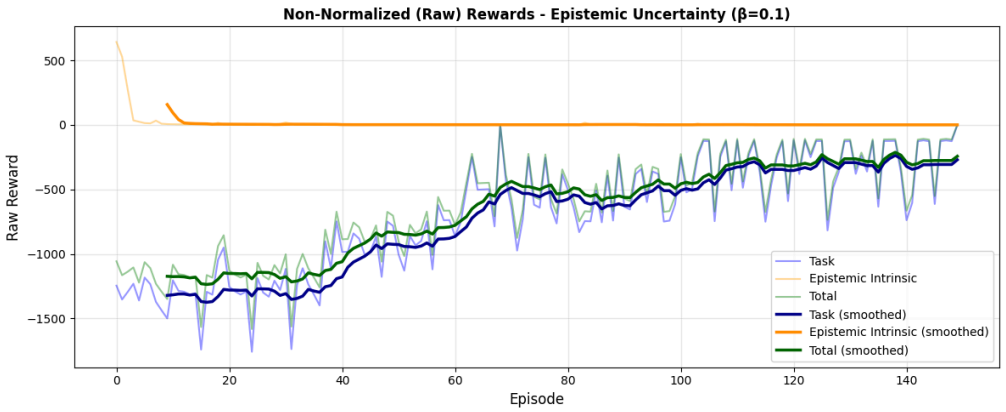
```python
# DDPG with Epistemic Uncertainty via Ensemble of Forward Models
# This solves the "noisy TV problem" by using ensemble disagreeme
nt instead of prediction error

import torch
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
import gymnasium as gym
import importlib# Import ensemble classes and training function f
rom helper module
import ddpg_intrinsic_helpers
importlib.reload(ddpg_intrinsic_helpers)
from ddpg_intrinsic_helpers import (
    Actor, Critic, ReplayBuffer, RunningNormalizer,
    ForwardModelEnsemble,
    select_action_with_noise, soft_update, plot_training_curves,
    train_ddpg_with_epistemic_uncertainty
)
```

In [ ]:

```
# Train on standard Pendulum
print("="*70)
print("DDPG WITH EPISTEMIC UNCERTAINTY (Standard Pendulum)")
print("="*70)
beta = 0.1
(actor_epistemic, ensemble_epistemic,
 task_rewards_epistemic, epistemic_rewards, total_rewards_epistem
ic,
 task_rewards_norm_epistemic, epistemic_rewards_norm, total_rewar
ds_norm_epistemic) = train_ddpg_with_epistemic_uncertainty(
    env_name='Pendulum-v1',
    beta=beta,
    noise_std=0.32,
    num_episodes=150,
    max_steps=200,
    n_ensemble=5
)

# Plot normalized rewards for epistemic uncertainty
plot_training_curves(
    data_list=[task_rewards_norm_epistemic, epistemic_rewards_nor
m, total_rewards_norm_epistemic],
    label_list=['Task', 'Epistemic Intrinsic', 'Total'],
    title=f'Normalized Rewards - Epistemic Uncertainty (β={bet
a})',
    ylabel='Normalized Reward',
    set_ylim_bottom=-0.2
)

# Plot raw rewards for epistemic uncertainty
plot_training_curves(
    data_list=[task_rewards_epistemic, epistemic_rewards, total_r
ewards_epistemic],
    label_list=['Task', 'Epistemic Intrinsic', 'Total'],
    title=f'Non-Normalized (Raw) Rewards - Epistemic Uncertainty
(β={beta})',
    ylabel='Raw Reward'
)


print("\n✓ Training completed!")
print("The agent uses ensemble disagreement (epistemic uncertaint
y) as intrinsic reward.")
print("This approach avoids the noisy TV problem by ignoring alea
toric uncertainty.")
```

```
=================================================================
=====
DDPG WITH EPISTEMIC UNCERTAINTY (Standard Pendulum)
=================================================================
=====
Training DDPG with epistemic uncertainty (β=0.1, ensemble=5)...
State dim: 3, Action dim: 1

Episode 20/150 | Task: -1297.34 | Epistemic: 81.9288
Episode 40/150 | Task: -1255.83 | Epistemic: 3.9020
Episode 60/150 | Task: -901.74 | Epistemic: 2.0673
Episode 80/150 | Task: -548.50 | Epistemic: 1.5345
Episode 100/150 | Task: -578.14 | Epistemic: 2.2256
Episode 120/150 | Task: -346.05 | Epistemic: 1.9316
Episode 140/150 | Task: -275.65 | Epistemic: 0.9293
```

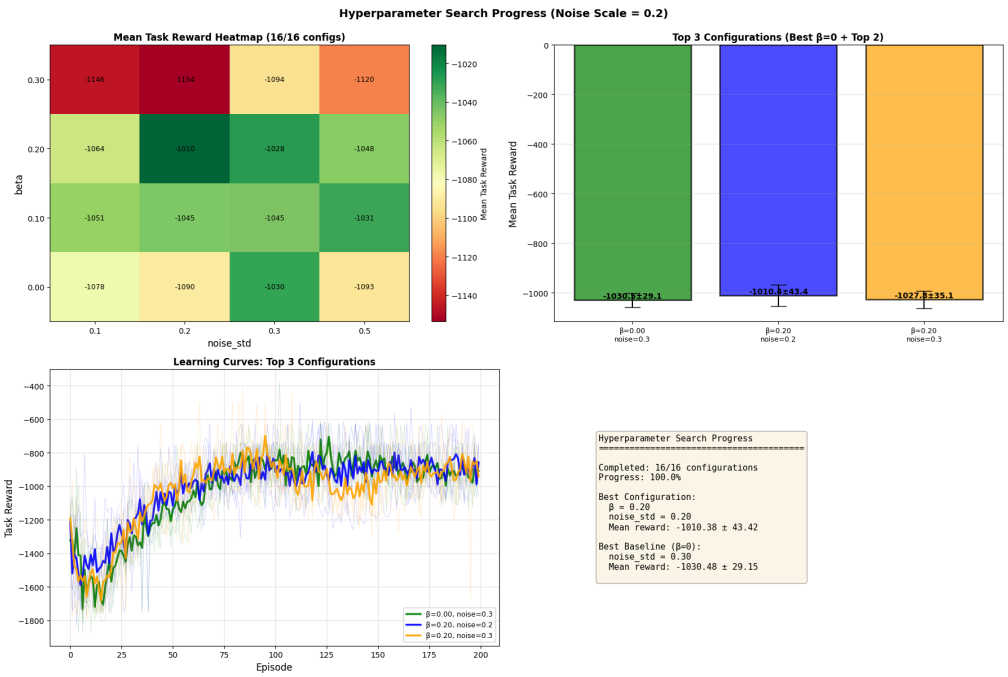





```
✓ Training completed!
The agent uses ensemble disagreement (epistemic uncertainty) as i
ntrinsic reward.
This approach avoids the noisy TV problem by ignoring aleatoric u
ncertainty.
```

In [8]:
```
# Hyperparameter Search: Epistemic Uncertainty with NoisyPendulum
# Compare epistemic uncertainty approach vs prediction error on n
oisy environments

import importlib
import ddpg_intrinsic_helpers
importlib.reload(ddpg_intrinsic_helpers)
from ddpg_intrinsic_helpers import (train_ddpg_with_epistemic_unc
ertainty,
                                    run_config_trials, plot_sear
ch_progress)
from functools import partial
import pandas as pd
from IPython.display import clear_output

# Use the already registered NoisyPendulum environment
# (registered in previous cell)

# Setup hyperparameter grid
beta_values = [0.0, 0.1, 0.2, 0.3]
noise_std_values = [0.1, 0.2, 0.3, 0.5]
num_trials = 5  # Run each configuration 7 times
num_episodes = 200  # Episodes per run

all_params = [{'beta': b, 'noise_std': ns} for b in beta_values f
or ns in noise_std_values]

# SET NOISE SCALE HERE (should match the one used above)
noise_scale = 0.2

print("\n" + "="*70)
print(f"HYPERPARAMETER SEARCH: EPISTEMIC UNCERTAINTY (NOISE_SCALE
= {noise_scale})")
print("="*70)

# Create a partial function for environment registration in worke
rs
register_env_worker = partial(register_noisy_pendulum_env, noise_
scale=noise_scale)

# Different cache directory for epistemic uncertainty experiments
cache_dir_epistemic = f'hyperparam_search/epistemic_noisy_pendulu
m_scale_{noise_scale}'

# Run experiments with live plotting and multiprocessing
results_epistemic = []

for idx, params in enumerate(all_params):
    beta = params['beta']
    noise_std = params['noise_std']

    # Run trials for this configuration (uses caching and multipr
ocessing)
    config_result = run_config_trials(
        train_ddpg_with_epistemic_uncertainty, beta, noise_std, n
um_trials, num_episodes,
```

```
        cache_dir=cache_dir_epistemic, env_name='NoisyPendulum-v
0',
        num_workers=5,  # Use 5 parallel workers
        env_registration_fn=register_env_worker  # Pass picklable
partial function
    )
    results_epistemic.append(config_result)

    clear_output(wait=True)

    # Convert current results to DataFrame
    df_results_epistemic = pd.DataFrame(results_epistemic)

    plot_search_progress(df_results_epistemic, idx, len(all_param
s),
                         noise_scale=noise_scale)

print("\n" + "="*70)
print("EPISTEMIC UNCERTAINTY HYPERPARAMETER SEARCH COMPLETE")
print("="*70)

# Compare best configurations from both approaches
best_pred_error = df_results.loc[df_results['mean_reward'].idxmax
()]
best_epistemic = df_results_epistemic.loc[df_results_epistemic['m
ean_reward'].idxmax()]

print(f"\nBest Prediction Error Config:")
print(f"  β={best_pred_error['beta']:.2f}, noise_std={best_pred_e
rror['noise_std']:.2f}")
print(f"  Mean reward: {best_pred_error['mean_reward']:.2f} ± {be
st_pred_error['std_reward']:.2f}")

print(f"\nBest Epistemic Uncertainty Config:")
print(f"  β={best_epistemic['beta']:.2f}, noise_std={best_epistem
ic['noise_std']:.2f}")
print(f"  Mean reward: {best_epistemic['mean_reward']:.2f} ± {bes
t_epistemic['std_reward']:.2f}")

print(f"\nImprovement: {best_epistemic['mean_reward'] - best_pred
_error['mean_reward']:.2f}")
```




```
=================================================================
=====
EPISTEMIC UNCERTAINTY HYPERPARAMETER SEARCH COMPLETE
=================================================================
=====

Best Prediction Error Config:
  β=0.00, noise_std=0.30
  Mean reward: -1012.89 ± 18.47

Best Epistemic Uncertainty Config:
  β=0.20, noise_std=0.20
  Mean reward: -1010.38 ± 43.42

Improvement: 2.51
```

Surprise and entropy

Entropy as a measure of surprise

Shannon Entropy quantifies the average uncertainty or surprise associated with a random variable. It measures the expected amount of information (in bits or nats) needed to describe the outcome of that variable.

Mathematical Definition:

For a discrete random variable X with possible outcomes $x_1, x_2, \ldots, x_n$ and probability mass function $p(x)$, the Shannon entropy $H(X)$ is defined as:

$$H(X) = -\sum_{i=1}^{n} p(x_i) \log p(x_i)$$

or more compactly:

$$H(X) = -\mathbb{E}_{x \sim p}[\log p(x)] = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

Convention: By convention, $0 \log 0 = 0$ (since $\lim_{p \to 0} p \log p = 0$).

Units:

- If $\log = \log_2$: entropy measured in **bits**
- If $\log = \ln$ (natural log): entropy measured in **nats**

Interpretation:

Property	Meaning
High Entropy	High uncertainty, maximum surprise, uniform distribution
Low Entropy	Low uncertainty, low surprise, peaked distribution
Maximum Entropy	$H(X) = \log n$ for uniform distribution over n outcomes
Minimum Entropy	$H(X) = 0$ for deterministic outcome (one $p(x_i) = 1$)

Entropy-regularized RL (Entropy as intrinsic reward)

Entropy as Intrinsic Reward in Actor-Critic

We can use policy entropy as an intrinsic reward to encourage exploration. The augmented reward combines task reward with entropy bonus:

$$r_{\text{total}} = (1-\beta) \cdot r_{\text{task}} + \beta \cdot H(\pi_\theta(\cdot|s))$$

where β controls the exploration-exploitation trade-off.

Algorithm: Actor-Critic with Entropy Intrinsic Reward

This shows the simple stochastic actor-critic algorithm with entropy as intrinsic motivation for exploration:

Algorithm: Actor-Critic with Entropy Bonus
Input: $\alpha_\theta, \alpha_\phi$ (with $\alpha_\phi >> \alpha_\theta), \gamma, \beta$
Output: $\pi_\theta(a|s), Q_\phi(s,a)$
---
1. Initialize actor parameters θ and critic parameters ϕ
2. For episode $= 1, 2, \ldots:$
  3. Observe initial state s_0
  4. For $t = 0, 1, 2, \ldots:$
    5. Sample action: $a_t \sim \pi_\theta(\cdot|s_t)$
    6. Execute a_t, observe task reward $r_{\text{task},t}$, next state s_{t+1}
    7. Compute policy entropy: $H_t = -\sum_a \pi_\theta(a|s_t) \log \pi_\theta(a|s_t)$
    8. Compute total reward: $r_{\text{total},t} = (1-\beta) \cdot r_{\text{task},t} + \beta \cdot H_t$
    9. Sample next action: $a_{t+1} \sim \pi_\theta(\cdot|s_{t+1})$
    10. Compute TD target: $y_t = r_{\text{total},t} + \gamma Q_\phi(s_{t+1}, a_{t+1})$
    11. Update critic: $\phi \leftarrow \phi - \alpha_\phi \cdot (Q_\phi(s_t, a_t) - y_t) \cdot \nabla_\phi Q_\phi(s_t, a_t)$
    12. Update actor: $\theta \leftarrow \theta + \alpha_\theta \cdot Q_\phi(s_t, a_t) \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)$
    13. If done, break
14. Return $\pi_\theta(a|s), Q_\phi(s,a)$

Key Features:

- **Line 7**: Computes entropy of policy distribution at current state
- **Line 8**: **Augments reward** with entropy bonus $\beta \cdot H_t$
  - High entropy → more exploration (diverse action probabilities)
  - Low entropy → more exploitation (concentrated on best actions)
- **Lines 10-12**: Standard actor-critic updates using total reward (task + entropy)
- Parameter β controls exploration-exploitation trade-off

Connection to Maximum Entropy RL:

This formulation is the foundation of maximum entropy reinforcement learning (MaxEnt RL), where the objective becomes:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t (r_{\text{task}}(s_t, a_t) + \beta H(\pi(\cdot|s_t))) \right]$$

The agent maximizes both cumulative task reward and policy entropy, leading to robust policies that maintain exploration throughout learning.

Soft Actor-Critic (SAC)

SAC augments reward with entropy: The Soft Actor-Critic (SAC) algorithm implements maximum entropy RL by augmenting the reward with an entropy term, enabling the agent to learn policies that are both reward-maximizing and highly stochastic.

[(COS435/ ECE 433: Introduction to RL - Lecture 17: Stochastic Actors and Maximum Entropy RL, Ben Eysenbach, 2025, p. 4-6, search for "Soft Actor Critic (SAC)") (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-17-maxent-rl.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-17-maxent-rl.pdf)

- Maximum entropy RL
- Augments reward with entropy
- Enables stochastic policy learning

Temperature parameter α in SAC: In SAC, the temperature parameter α determines the relative importance of the entropy term; tuning α allows practitioners to control the balance between exploration (high entropy) and exploitation (high reward).

[(COS435/ ECE 433: Introduction to RL - Lecture 17: Stochastic Actors and Maximum Entropy RL, Ben Eysenbach, 2025, p. 4-6, search for "choosing alpha is that it depends on the reward scale") (https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-17-maxent-rl.pdf)](https://ben-eysenbach.github.io/intro-rl/s25_material/lecture-17-maxent-rl.pdf)

- Temperature parameter α
- Controls exploration vs. exploitation
- Tuning affects entropy's importance

The Soft Actor-Critic algorithm (Pseudocode)

Algorithm: Soft Actor-Critic (SAC)

Based on [OpenAI Spinning Up - SAC (https://spinningup.openai.com/en/latest/algorithms/sac.html)](https://spinningup.openai.com/en/latest/algorithms/sac.html)

Input: Initial policy parameters θ, Q-function parameters ϕ_1, ϕ_2, empty replay buffer
Output: Trained policy π_θ and Q-functions Q_{ϕ_1}, Q_{ϕ_2}

1. Set target parameters equal to main parameters:
$\phi_{\text{targ},1} \leftarrow \phi_1, \quad \phi_{\text{targ},2} \leftarrow \phi_2$

2. Repeat until convergence:
3. Observe state s and select action $a \sim \pi_\theta(\cdot|s)$
4. Execute a in the environment
5. Observe next state s', reward r, and done signal done
6. Store $(s, a, r, s', \text{done})$ in replay buffer $\mathcal{D}$
7. If done, reset environment state

8. If it's time to update:
9. For j in range(however many updates):
10. Randomly sample a batch of transitions $B = \{(s, a, r, s', \text{done})\}$ from $\mathcal{D}$

11. Compute targets for the Q functions (KEY STEP):

$$y(r, s', \text{done}) = r + \gamma(1 - \text{done}) \left(\min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s') \right)$$

where $\tilde{a}' \sim \pi_\theta(\cdot|s')$

Note: Entropy term $-\alpha \log \pi_\theta(\tilde{a}'|s')$ is added to Q-value, not as intrinsic rew
Effect is similar to entropy-regularized AC above, but applied in target comp

12. Update Q-functions by one step of gradient descent using:

$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,r,s',\text{done}) \in B} \left(Q_{\phi_i}(s, a) - y(r, s', \text{done}) \right)^2 \quad \text{for } i = 1, 2$$

13. Update policy by one step of gradient ascent using:

$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} \left(\min_{i=1,2} Q_{\phi_i}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s)|s) \right)$$

where $\tilde{a}_\theta(s)$ is sampled from $\pi_\theta(\cdot|s)$ (reparameterization trick)

14. Soft update target networks:
$\phi_{\text{targ},i} \leftarrow \tau_{\text{soft}} \phi_i + (1 - \tau_{\text{soft}}) \phi_{\text{targ},i} \quad \text{for } i = 1, 2$

Key Features:

Component	Description
Entropy Regularization	Policy maximizes $\mathbb{E}[Q(s,a) - \alpha \log \pi(a\|s)]$, encouraging exploration
Clipped Double-Q	Uses minimum of two Q-functions to reduce overestimation bias (line 11, 13)
Stochastic Policy	Squashed Gaussian policy: $\tilde{a} = \tanh(\mu_\theta(s) + \sigma_\theta(s) \odot \xi)$ where $\xi \sim \mathcal{N}(0, I)$ is random noise
Reparameterization Trick	Separates randomness (ξ) from parameters (θ): enables gradient flow through stochastic sampling (line 13)
Target Networks	Stabilizes training via soft update with $\tau_{\text{soft}} \approx 0.005$ (line 14)
Temperature α	Controls exploration-exploitation trade-off; can be fixed or learned

Reparameterization Trick Explained:

The **reparameterization trick** allows gradients to flow through stochastic sampling by separating the randomness from the parameters:

- **Without reparameterization**: $a \sim \pi_\theta(\cdot|s) \rightarrow$ cannot differentiate through sampling
- **With reparameterization**: $a = \pi_\theta(s, \xi)$ where $\xi \sim \mathcal{N}(0, I) \rightarrow$ can differentiate w.r.t. θ

For SAC's **squashed Gaussian policy**:

1. Network outputs mean $\mu_\theta(s)$ and log-std $\log \sigma_\theta(s)$
2. Sample noise: $\xi \sim \mathcal{N}(0, I)$ (independent of θ)
3. Compute pre-squashed action: $u = \mu_\theta(s) + \sigma_\theta(s) \odot \xi$ (element-wise multiplication $\odot$)
4. Apply squashing: $\tilde{a} = \tanh(u)$ (bounds actions to $[-1, 1]$)

This formulation allows $\nabla_\theta \tilde{a}_\theta(s)$ to exist, enabling policy gradient computation via chain rule.

Differences from DDPG/TD3:

- **DDPG/TD3**: Deterministic policy with added noise for exploration
- **SAC**: Stochastic policy with entropy bonus for principled exploration
- **SAC advantage**: More robust, better exploration, often higher sample efficiency

[(OpenAI Spinning Up - SAC, 2018) (https://spinningup.openai.com/en/latest/algorithms/sac.html)](https://spinningup.openai.com/en/latest/algorithms/sac.html)

Proximal Policy Optimization (PPO)

(was skipped in lecture, not relevant for exam)

PPO as a practical policy gradient method: Proximal Policy Optimization (PPO) is a widely-used policy gradient algorithm that improves upon vanilla policy gradients by constraining policy updates to avoid destructively large changes, leading to more stable and sample-efficient learning.

Key insight - clipped objective: Instead of enforcing hard constraints on policy updates (like its predecessor TRPO), PPO uses a clipped surrogate objective that penalizes updates that move the policy too far from its previous version, making it simpler to implement while maintaining stability.

(OpenAI Spinning Up - PPO, 2018) (https://spinningup.openai.com/en/latest/algorithms/ppo.html)

- Policy gradient algorithm
- Constrains policy updates to avoid large changes
- More stable than vanilla policy gradients
- Simpler than TRPO (Trust Region Policy Optimization)

Entropy regularization in PPO: Like SAC, PPO can include an entropy bonus to encourage exploration, making it suitable for environments with sparse rewards or where maintaining diverse behaviors is beneficial.

(OpenAI Spinning Up - PPO, 2018, section "Entropy Regularization Bonus") (https://spinningup.openai.com/en/latest/algorithms/ppo.html#exploration-vs-exploitation)

- Entropy coefficient c_1 controls exploration
- Prevents premature convergence to deterministic policies
- Similar to SAC's temperature parameter

PPO supports discrete actions: Unlike SAC (which uses continuous Gaussian policies), PPO naturally handles discrete action spaces through categorical (softmax) policies, making it ideal for Atari games and other discrete control tasks.

- Works with both discrete and continuous actions
- Categorical policies for discrete actions (Atari)
- Gaussian policies for continuous actions (robotics)

The Proximal Policy Optimization algorithm (Pseudocode)

Algorithm: Proximal Policy Optimization (PPO-Clip)

Based on [OpenAI Spinning Up - PPO](https://spinningup.openai.com/en/latest/algorithms/ppo.html) (https://spinningup.openai.com/en/latest/algorithms/ppo.html)

Input: Initial policy parameters θ_0, value function parameters ϕ_0
Output: Trained policy π_θ and value function V_ϕ

1. For $k = 0, 1, 2, \ldots$ **do:**

2. Collect trajectory set $\mathcal{D}_k = \{\tau_i\}$ by running policy π_{θ_k} in the environment
Each trajectory $\tau_i = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$

3. Compute advantages $\hat{A}_t$ using Generalized Advantage Estimation (GAE), as follo
$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \ldots$
where $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ (TD residual)
and $\lambda \in [0, 1]$ controls bias-variance trade-off (typically $\lambda \approx$ 0.95-0.99)

4. Compute returns $\hat{g}_t$ (rewards-to-go):

$$\hat{g}_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$$

5. For epoch $e = 1, \ldots, E$ **do:**

6. For minibatch $B \subseteq \mathcal{D}_k$ **do:**

7. Compute probability ratio:

$$\kappa_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}$$

8. Compute clipped surrogate objective (KEY STEP):

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min\left(\kappa_t(\theta)\hat{A}_t,\ \text{clip}(\kappa_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$$

where ϵ is the clip range (typically 0.1-0.2)
Prevents large policy updates that could destabilize training

$$L^V(\phi) = \mathbb{E}_t \left[(V_\phi(s_t) - \hat{g}_t)^2\right]$$
$$L^V(\phi) = \mathbb{E}_t \left[(V_\phi(s_t) - \hat{R}_t)^2\right]$$

10. Compute entropy bonus:

$$S[\pi_\theta](s_t) = \mathbb{E}_t \left[H(\pi_\theta(\cdot|s_t))\right] = \mathbb{E}_t \left[-\sum_a \pi_\theta(a|s_t) \log \pi_\theta(a|s_t)\right]$$

11. Update policy and value function:
$\theta \leftarrow \theta + \alpha \nabla_\theta L^{\text{CLIP}}(\theta)$
$\phi \leftarrow \phi - \alpha \nabla_\phi L^V(\phi)$
(Entropy bonus $S[\pi_\theta]$ can be added with coef $c_2 \approx 0.01$ for exploration)

Key Features:

Component	Description
Clipped Objective	Prevents large policy updates via $\text{clip}(\kappa_t, 1-\epsilon, 1+\epsilon)$; maintains training stability (line 8)
Generalized Advantage Estimation (GAE)	Computes advantages as weighted sum of TD residuals with $\lambda \in [0, 1]$ controlling bias-variance trade-off (line 3)
Multiple Epochs	Reuses collected data for multiple gradient updates, improving sample efficiency (line 5)
Entropy Bonus	Encourages exploration; can be added to policy update with coefficient $c_2 \approx 0.01$ (line 11)
Value Function Loss	Learns state-value function for advantage estimation; trained jointly with policy (line 9)
Works with Discrete & Continuous Actions	Categorical policy for discrete, Gaussian for continuous

Clipped Objective Explained:

The **clipped surrogate objective** is PPO's key innovation:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\kappa_t(\theta) \hat{A}_t,\ \text{clip}(\kappa_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]$$

- **Probability ratio**: $\kappa_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ measures how much the policy changed
- **Clipping**: Limits κ_t to $[1-\epsilon, 1+\epsilon]$ (typically $\epsilon = 0.2$)
- **Effect**:
  - If advantage is positive ($\hat{A}_t > 0$): don't increase probability too much (clip at $1+\epsilon$)
  - If advantage is negative ($\hat{A}_t < 0$): don't decrease probability too much (clip at $1-\epsilon$)
- **Result**: Conservative policy updates that prevent catastrophic performance drops

Comparison with SAC:

Aspect	PPO	SAC
Action Space	Discrete OR Continuous	Continuous only
Policy Type	Stochastic (categorical/Gaussian)	Stochastic (Gaussian)
Update Constraint	Clipped objective	Entropy regularization in Q-target
Exploration	Entropy bonus (optional)	Entropy bonus (built-in)
Sample Efficiency	On-policy (lower)	Off-policy (higher)
Use Cases	Atari games, robotics	Continuous control, robotics

Both PPO and SAC implement maximum entropy RL principles, but PPO's discrete action support makes it ideal for Atari games like Montezuma's Revenge.

[(OpenAI Spinning Up - PPO, 2018) (https://spinningup.openai.com/en/latest/algorithms/ppo.html)
(OpenAI Spinning Up - PPO, 2018) (https://spinningup.openai.com/en/latest/algorithms/ppo.html)

Summary

This lecture explored exploration techniques and intrinsic motivation mechanisms that enable RL agents to discover optimal policies in complex environments.

Key Concepts:

Exploration and intrinsic motivation:

- Curiosity-driven exploration augments task rewards with intrinsic rewards
- Intrinsic motivation enables learning without external rewards
- Balance factor β controls exploration-exploitation trade-off

Uncertainty types:

- **Epistemic uncertainty**: Reducible through data collection (model disagreement)
- **Aleatoric uncertainty**: Irreducible randomness (inherent stochasticity)
- Ensembles quantify epistemic uncertainty via prediction variance

Prediction error as intrinsic reward:

- Forward model prediction error $||s' - f_\psi(s, a)||^2$ encourages exploration
- High prediction error indicates novel/uncertain states
- **Noisy TV problem**: Agents attracted to stochastic states without learning value

Information-theoretic exploration:

- Surprise: Negative log-likelihood $-\log p(s'|s, a)$ rewards rare transitions
- Entropy regularization: $H(\pi(\cdot|s))$ encourages stochastic policies
- SAC maximizes $\mathbb{E}[r + \alpha H(\pi)]$ for principled exploration

Advanced algorithms:

- **Entropy-regularized Actor-Critic**: On-policy with entropy bonus
- **Soft Actor-Critic (SAC)**: Off-policy with automatic entropy tuning and soft updates
- **Proximal Policy Optimization (PPO)**: On-policy with entropy regularization and policy change surrogate objective

Practical insights:

- Prediction error exploration can fail in stochastic environments
- Entropy regularization provides robust exploration in continuous control
- Proper uncertainty quantification distinguishes valuable exploration from noise

Outlook

From exploration to structured goal-directed learning:

The exploration techniques covered in this lecture—curiosity-driven intrinsic rewards, prediction error bonuses, and entropy regularization—help agents discover diverse behaviors and visit novel states. However, these methods lack explicit goal structure: they explore broadly but don't systematically learn to reach specific targets.

Limitations of current exploration approaches:

- Intrinsic rewards are unstructured (reward novelty, not goal achievement)
- No explicit representation of desired outcomes or goals
- Difficult to leverage exploration for multi-goal tasks
- Learning is inefficient when agent must achieve specific objectives

Next lecture: Reward-shaping and Goal-conditioned RL

The next lecture addresses these limitations by introducing techniques that incorporate explicit goal structure into the learning process:

- **Reward shaping**: Modify reward function to guide learning while preserving optimal policy
- **Universal Value Function Approximation**: Learn value functions conditioned on goals, enabling generalization across multiple objectives
- **Hindsight Experience Replay**: Convert failed experiences into successful ones by relabeling goals, dramatically improving sample efficiency

These methods enable agents to learn goal-directed behaviors efficiently, leveraging exploration to achieve specific targets rather than aimless novelty-seeking.